Memoria de prácticas Campus Rural 2025-2026

Archmundi: Archivo digital Municipal para el ayuntamiento de Albocasser (Castellón)

Adrián Guerras Algarra

1. Introducción y objetivos

En este proyecto buscaba construir una plataforma en forma de aplicación web que actuase como archivo municipal digital para el ayuntamiento de la localidad de Albocasser (Castellón). Cualquier persona puede consultar y descargar el contenido sin necesidad de iniciar sesión ni identificarse; aunque únicamente los administradores pueden añadir, editar y eliminar las entradas.

El modelo de datos de la aplicación se basa en entradas, una entidad abstracta que cuenta con un identificador interno, un nombre, un año y etiquetas, de la cual heredan las tres clases definidas para los tipos de contenido que acepta la plataforma.

Objetivos principales

- Permitir la consulta pública del archivo municipal, organizando y filtrando por tipo de contenido, año o etiquetas.
- Restringir la gestión del contenido al personal del ayuntamiento.
- Almacenar de forma ordenada los documentos relacionados con la localidad.

2. Arquitectura y tecnologías

Para construir la aplicación, se ha utilizado una arquitectura de 3 capas desacopladas: el frontend, la API y la capa de persistencia. Una decisión de diseño, dado el volumen de datos, fue no guardar los archivos binarios en la base de datos, en la que se guardan únicamente los metadatos y referencias al archivo. Gracias a esta decisión, se evitan las copias de seguridad enormes, las consultas lentas y problemas a la hora de servir vídeos.

2.1. Backend

- Node.js con Express (API REST).
- SQL Server (T-SQL) con el driver mssql.
- JWT (jsonwebtoken) para la autenticación y bcrypt para el almacenamiento seguro de contraseñas.
- Multer para la recepción de archivos subidos.
- fluent-ffmpeg y ffmpeg-static para generar miniaturas de vídeo.
- cors y dotenv para la configuración.

2.2. Frontend

- Angular con componentes standalone, signals y detección de cambios sin Zone.js. El proyecto tiene renderizado en servidor (SSR) activado.
- Angular Router con un guard funcional que protege las rutas de administración.
- HttpClient con un interceptor funcional que añade el token y cierra la sesión si éste caduca.
- CSS propio con variables y tipografías Nunito y Source Sans 3.

Herramientas de desarrollo: Visual Studio Code, SQL Server Management Studio y Postman.

Prototipo alternativo: en paralelo se desarrolló una versión del frontend en React con Vite, que se descartó al decidir continuar con Angular.

3. Trabajo realizado

3.1. Base de datos

Para la base de datos se diseñó un esquema relacional en SQL Server con un total de siete tablas:

- **Entradas**: tabla base con los campos comunes (identificador, nombre, año, tipo, descripción, fechas y autor).
- **Fotos**, **Documentos** y **Vídeos**: tablas hijas que tienen una relación 1:1 con Entradas, cada una con sus campos específicos (ruta al archivo, formato, dimensiones, hash, o páginas y miniatura, en caso de tratarse de un documento de texto o vídeo, respectivamente).
- **Etiquetas** y **EntradaEtiqueta**: etiquetas normalizadas y la relación muchos a muchos con las entradas.
- **Administradores**: usuarios con la contraseña almacenada como hash.

Elementos adicionales: identificadores GUID para evitar su enumeración, restricciones `CHECK` (tipo y rango de años), borrado en cascada, una vista unificada, `VistaEntradas`, para el listado público, un trigger que mantiene la fecha de actualización y un segundo trigger que limita la jerarquía de etiquetas a dos niveles. Los scripts son idempotentes (es decir, que se pueden ejecutar varias veces sin error) y se acompañan de un script de pruebas.

3.2. API REST

- Autenticación: `POST /api/auth/login`.
- Entradas, públicas: `GET /api/entradas` (con filtros por tipo, etiqueta, año y texto) y `GET /api/entradas/:id`.
- Entradas, solo administrador: `POST`, `PUT` y `DELETE`.
- Etiquetas: `GET /api/etiquetas` (árbol de categorías y subcategorías, público) y `POST`/`DELETE` (administrador).
- Comprobación de estado: `GET /api/health`.

Cabe destacar que se han implementado las siguientes características:

- Consultas parametrizadas para prevenir inyección SQL.
- Mensaje de error uniforme en el login de administrador.
- Operaciones de escritura en transacciones.
- Validación de la extensión del archivo según el tipo de entrada.
- Generación automática de miniaturas para vídeos a partir del primer fotograma.
- Borrado del archivo antiguo sólo después de confirmar la transacción.
- Normalización de las etiquetas en minúsculas para evitar duplicados.

3.3. Frontend (Angular)

En el frontend se han desarrollado los siguientes elementos:

- **Inicio**: página principal que cuenta con cinco accesos según la necesidad del cliente (Fotos, Vídeos, Fiestas y Asociaciones).
- **Listado**: lista de entradas por tipo de contenido o por etiqueta, mostradas en cuadrículas con miniatura o icono y datos relevantes.
- **Detalle**: permite ver el documento en detalle, leer información adicional, descargarlo o, en caso de ser un vídeo, reproducirlo.
- **Cabecera**: elemento reutilizable que se muestra en todas las páginas y que cuenta con el logo (que devuelve a la página de inicio), un buscador y un enlace al login/logout.
- **Login de administrador**: inicio de sesión con sesión persistente hasta cerrar la sesión o caducar el token.
- **Panel de administración**: tabla con todas las entradas de la plataforma, con ordenación por año y tipo, y con acciones de ver, editar y eliminar.
- **Formulario de entrada**: para crear y editar las entradas que cuenta con un selector de etiquetas limitado a las que están registradas, inclusión automática de la categoría padre al seleccionar una etiqueta (si por ejemplo se selecciona la categoría “fin de año”, se le añade de forma automática la categoría de “fiestas”), y mensaje de error en una ventana modal
- **Gestión de etiquetas**: una página en la que se pueden crear y borrar categorías y subcategorías.

3.4. Pruebas

Se verificó el esquema con un script SQL (inserción, vista, triggers, restricciones y cascada) y se probaron todos los endpoints con Postman, incluidos los casos sin token y los archivos de tipo incorrecto. El resto de

comprobaciones se hicieron manualmente en el navegador. No existen aún pruebas automatizadas.

3.5. Incidencias técnicas relevantes

- *localStorage* no existe en el servidor, lo que rompía el SSR: se añadió una comprobación del entorno de ejecución.
- Al ser un proyecto sin Zone.js, el estado asíncrono debe gestionarse con signals; de lo contrario, la pantalla no se actualiza.
- Las versiones recientes de Angular CLI nombran los archivos sin el sufijo `.component` y usan la carpeta *public* para los recursos estáticos.
- Varios comportamientos inconsistentes se debieron a la caché del navegador y de *ng serve*.

4. Estado actual

Actualmente la aplicación es funcional en entorno local en sus aspectos básicos. Quedan sin implementar en Angular varias partes del prototipo y de la cabecera:

- Páginas de categoría con acordeón por subcategoría y año, y vistas con ruta de navegación (por ejemplo, Vídeos/Asociación de fútbol/2026).
- Búsqueda global: la cabecera navega a /buscar, pero esa ruta no existe todavía.
- Página de contacto
- Selector de idioma ES/VA
- Enlace de accesibilidad.
- Miniaturas para documentos.

5. Pasos pendientes para el despliegue en Azure

5.1. Cambios de código previos

- Migrar el almacenamiento de archivos a Azure Blob Storage: el disco de App Service no es persistente. Multer pasará a memoryStorage, se creará un módulo de almacenamiento, se guardará la URL completa del archivo y se borrarán los blobs antiguos al editar o eliminar.
- Actualizar el frontend: usar RutaArchivo directamente (ya es una URL completa), retirar el proxy de desarrollo y definir la URL de la API en un archivo de entorno de producción.
- Restringir CORS al dominio real del frontend y eliminar la ruta estática /uploads.
- Valorar si se mantiene el SSR: Static Web Apps sirve la aplicación como contenido estático; si no se prioriza el SEO, desactivarlo simplifica el despliegue.
- Revisar los límites de memoria: con memoryStorage, subidas de hasta 500 MB pueden agotar la memoria del plan gratuito. Conviene reducir el límite o subir directamente a Blob Storage.
- Probar ffmpeg en App Service para Linux, ya que los recursos del plan gratuito son limitados.

5.2. Despliegue

- Crear la cuenta de Azure y un grupo de recursos.
- Crear Azure SQL Database con la oferta gratuita, configurar el firewall y ejecutar los scripts del esquema desde SSMS.
- Crear una cuenta de Storage con un contenedor de acceso público de lectura para los archivos.
- Crear el App Service (Node, Linux, plan F1), configurar las variables de entorno (base de datos, JWT, Storage) y desplegarlo desde VS Code.
- Subir el proyecto a GitHub y crear un Static Web App conectado al repositorio para el frontend.
- Crear el primer administrador contra la base de datos de Azure y probar el flujo completo.
- Configurar alertas de presupuesto desde el primer día: los límites gratuitos no bloquean el gasto automáticamente.

5.3. Actualizar la aplicación una vez desplegada

- Frontend: cada push a la rama principal dispara el flujo de GitHub Actions que compila y despliega.
- Backend: volver a desplegar desde VS Code con la extensión de Azure App Service o configurar un despliegue continuo desde GitHub.
- Base de datos: no volver a ejecutar el esquema completo; escribir scripts de migración numerados (por ejemplo 002_añadir_campo.sql), probarlos primero en local y ejecutarlos contra Azure SQL desde SSMS, añadiendo antes la IP al firewall. Antes de cambios destructivos, comprobar que existe una copia de seguridad reciente.
- Configuración: las variables de entorno se modifican en la sección de configuración del App Service.

5.4. Consideraciones normativas y de costes

Al tratarse de una administración pública, ha sido necesario hacer una consulta para que se licite la licencia de Azure, consultar el RGPD y los requisitos de accesibilidad de los sitios web del sector público. El problema con Azure reside en que, aunque Azure cuente con planes gratuitos, se debe hacer un contrato público.

6. Sugerencias de mejora

Seguridad

- Limitar los intentos de login y añadir cabeceras de seguridad (por ejemplo con helmet).
- Guardar el token en cookies httpOnly en lugar de localStorage, y valorar tokens de refresco y doble factor de autenticación.
- Validar el tipo real del archivo por su contenido y no solo por la extensión; analizar los archivos subidos en busca de malware.
- Gestionar los administradores desde la propia aplicación, con roles y registro de auditoría de las acciones.

Funcionalidad

- Completar las pantallas pendientes del prototipo (categorías con acordeón, rutas de navegación, búsqueda, contacto, idioma).
- Permitir renombrar y mover etiquetas, y mostrar el recuento de entradas por etiqueta.
- Miniaturas de la primera página de los PDF y versiones reducidas de las fotos.
- Paginación y búsqueda de texto completo para cuando el archivo crezca.
- Reproducción de vídeo con soporte de rangos HTTP y, si hay mucho volumen, procesamiento asíncrono mediante una cola.
- Metadatos adicionales (ubicación, autoría, licencia) y borrado lógico con papelera.

Rendimiento y operación

- CDN delante del almacenamiento, compresión de imágenes y subida directa a Blob Storage mediante URLs firmadas.
- Copias de seguridad programadas y monitorización (por ejemplo, Application Insights).
- Escalar a un plan de pago cuando aumente el tráfico o el volumen de archivos.
- Calidad y mantenimiento
- Pruebas automatizadas (API y componentes) y despliegue continuo también para el backend.

- Migraciones de base de datos versionadas y documentación de la API.
- Auditoría de accesibilidad (teclado, lectores de pantalla, contraste) y traducción real a valenciano.

7. Conclusiones

Se ha construido una base sólida y funcional: una base de datos bien normalizada, una API segura con roles diferenciados y un frontend que cubre la consulta pública y administración del contenido. Lo más urgente antes de publicar es migrar el almacenamiento de archivos a Blob Storage, completar algunas pantallas pendientes del prototipo y revisar la normativa de accesibilidad y el RGPD con el ayuntamiento. Con esos pasos el despliegue en Azure es viable, pudiendo llegar a mantenerse temporalmente dentro de los niveles del plan gratuito de Azure.